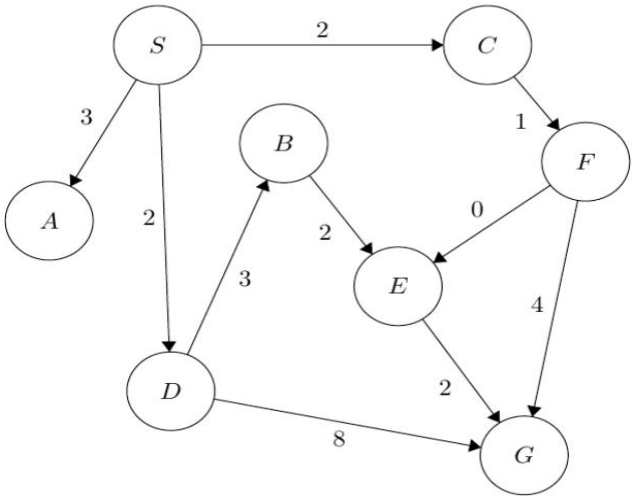
BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Phạm Đức Lương
MSSV / Lớp	3124720042 / DNT1241

Quy ước: goal test khi lấy node ra khỏi frontier; "Explored Set" ghi theo thứ tự node được lấy ra; hòa thì phá theo thứ tự alphabet.

Bài 1. Các thuật toán tìm kiếm



Các cạnh: S→A (3), S→C (2), S→D (2), C→F (1), D→B (3), D→G (8), B→E (2), F→E (0), F→G (4), E→G (2). Nút A không có cạnh đi ra.

1.1. Breadth-First Search (BFS)

Nội dung	**Kết quả**
Explored Set	S, A, C, D, F, B, G
Path	S → D → G
Total Cost	2 + 8 = 10

1.2. Depth-First Search (DFS)

Nội dung	**Kết quả**
Explored Set	S, A, C, F, E, G
Path	S → C → F → E → G
Total Cost	2 + 1 + 0 + 2 = 5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	**Các nút được duyệt**
0	S
1	S, A, C, D

Giới hạn độ sâu	Các nút được duyệt
2	S, A, C, F, D, B, G (tìm thấy G)

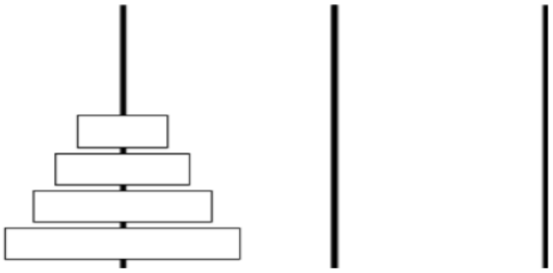
Path cuối cùng: S → D → G

Total Cost: 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

- **Frontier:** BFS dùng hàng đợi FIFO nên duyệt theo từng tầng. DFS dùng ngăn xếp LIFO nên đi sâu hết nhánh rồi mới quay lui.
- **Tính tối ưu:** BFS tối ưu về số bước khi các cạnh có chi phí bằng nhau, nhưng không tối ưu về chi phí khi cạnh có trọng số khác nhau. Ở đây BFS cho chi phí 10 trong khi DFS tìm được đường chi phí 5. DFS không đảm bảo tối ưu, và có thể không hoàn chỉnh nếu đồ thị có chu trình hoặc vô hạn mà không kiểm tra lặp. Bù lại, DFS tốn ít bộ nhớ hơn nhiều.

Bài 2. Tháp Hà Nội – Tower of Hanoi



2.1. Biểu diễn trạng thái

Một trạng thái là bộ n phần tử $s = (p_1, p_2, \ldots, p_n)$, với $p_i \in \{1, 2, 3\}$ là cọc chứa đĩa i (đĩa 1 nhỏ nhất, đĩa n lớn nhất). Thứ tự đĩa trên mỗi cọc luôn được xác định (lớn dưới, nhỏ trên), nên biểu diễn này đủ để xác định các hành động hợp lệ.

2.2. Kích thước không gian trạng thái

Mỗi đĩa độc lập chọn 1 trong 3 cọc, và mọi cách xếp đều hợp lệ vì trên mỗi cọc các đĩa luôn tự sắp theo kích thước. Vậy có 3^n trạng thái.

2.3. Trạng thái bắt đầu

$S_0 = (1, 1, \ldots, 1)$, tức tất cả đĩa ở cọc 1.

2.4. Các hành động hợp lệ

Actions(s) = {move(i, a→b)}, với $a = p_i$ và $b \neq a$, thỏa mãn:

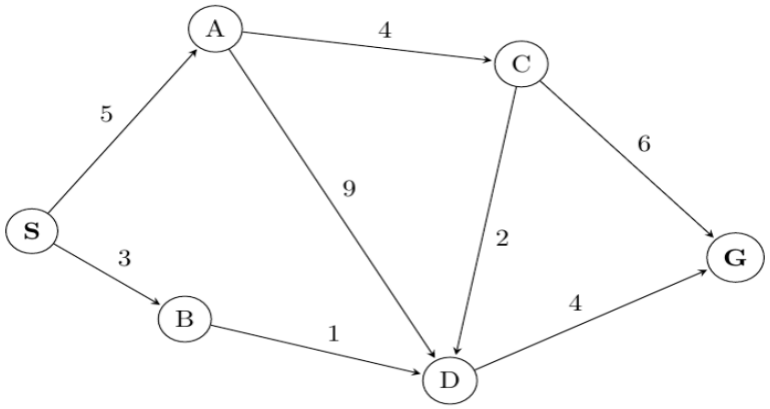
1. Đĩa i đang ở cọc a và là đĩa trên cùng của cọc đó, tức không có đĩa $j < i$ nào có $p_j = a$.
2. Đĩa i đặt được lên cọc b, tức không có đĩa $j < i$ nào có $p_j = b$ (b rỗng hoặc đĩa trên cùng của b lớn hơn i).
3. Mỗi lần chỉ chuyển một đĩa.

Mỗi trạng thái có tối đa 3 hành động hợp lệ.

2.5. Goal Test

goalTest(s) trả về True khi và chỉ khi $p_i = 3$ với mọi i = 1..n, tức s = (3, 3, …, 3). Số bước tối thiểu là $2^n - 1$.

Bài 3. Thiết kế và phân tích Heuristic



Các cạnh: S→A (5), S→B (3), A→C (4), A→D (9), B→D (1), C→D (2), C→G (6), D→G (4).

Chi phí thật h*: G = 0, D = 4, C = 6, B = 5, A = 10, S = 8.

3.1. Thiết kế heuristic admissible và consistent

Node	h(n)
S	7
A	9
B	4
C	5
D	3
G	0

- **Admissible:** mọi h(n) ≤ h*(n).
- **Consistent:** S→A: 7 ≤ 5+9. S→B: 7 ≤ 3+4. A→C: 9 ≤ 4+5. A→D: 9 ≤ 9+3. B→D: 4 ≤ 1+3. C→D: 5 ≤ 2+3. C→G: 5 ≤ 6+0. D→G: 3 ≤ 4+0. Tất cả đều thỏa.

3.2. Chạy A* với heuristic đã thiết kế

- Mở S: A có f = 5+9 = 14, B có f = 3+4 = 7.
- Mở B: D có g = 4, f = 7.
- Mở D: G có g = 8, f = 8.
- Lấy G (f = 8) là đích.

Explored Set: S, B, D, G

Path: S → B → D → G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

h(S) = 8, h(A) = 10, h(B) = 5, h(C) = 6, h(D) = 1, h(G) = 0.

- Admissible vì mọi h ≤ h* (D: 1 ≤ 4).
- Cạnh vi phạm: C→D có h(C) = 6 > c(C,D) + h(D) = 2 + 1 = 3. Cạnh B→D cũng vi phạm: 5 > 1 + 1 = 2.

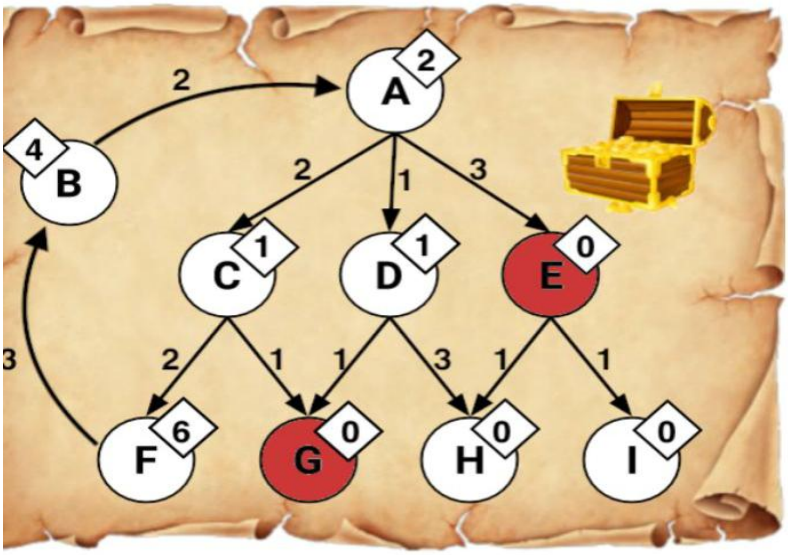
3.4. Câu hỏi mở rộng: consistent ⇒ admissible

Lấy đường đi tối ưu $n = n_0 \to n_1 \to \dots \to n_k = G$. Áp dụng consistency liên tiếp:

$$h(n_0) \le c(n_0,n_1) + h(n_1) \le c(n_0,n_1) + c(n_1,n_2) + h(n_2) \le \dots \le \Sigma c + h(G) = \Sigma c + 0 = h^*(n).$$

Vậy h(n) ≤ h*(n), tức admissible.

Bài 4. Treasure Hunting (Tìm kho báu)



Các cạnh: A→C (2), A→D (1), A→E (3), B→A (2), C→F (2), C→G (1), D→G (1), D→H (3), E→H (1), E→I (1), F→B (3). Kho báu ở E và G (nút đỏ). Heuristic: A = 2, B = 4, C = 1, D = 1, E = 0, F = 6, G = 0, H = 0, I = 0.

Thuật toán	Explored Set	Path returned	Cost
4.1 DFS	A, C, F, B, G	A → C → G	3
4.2 BFS	A, C, D, E	A → E	3
4.3 UCS	A, D, C, G	A → D → G	2
4.4 Greedy	A, E	A → E	3
4.5 A*	A, D, G	A → D → G	2

Ghi chú:

- **UCS:** sau khi mở A và D, frontier có C (2) và G (2) hòa nhau, nên chọn C theo alphabet. Mở C chỉ cho G với g = 3, kém hơn g = 2 đã có. Sau đó lấy G.
- **A*:** f(D) = 1 + 1 = 2, f(C) = 3, f(E) = 3. Lấy D, rồi G có f = 2 và là đích

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu lớn nhất	A→C→G	3
BFS	Độ sâu nhỏ nhất	A→E	3
UCS	g(n)	A→D→G	2
Greedy	h(n)	A→E	3
A*	g(n)+h(n)	A→D→G	2

Chỉ UCS và A* cho đường tối ưu (cost 2).

Bài 5. Câu hỏi Đúng/Sai

Câu 1 trong đề bị lỗi chữ ("ít nhất nhiều"), được hiểu là "DFS luôn mở rộng không nhiều hơn A*".

Câu	Đúng / Sai	Giải thích
Câu 1	Sai	DFS có thể đi vào nhánh sâu sai trước, còn A* với heuristic tốt đi thẳng tới đích. Ví dụ ở Bài 4, DFS mở 5 node còn A* chỉ mở 3. Nếu hiểu ngược lại ("DFS mở ít nhất bằng A*") thì cũng sai, vì DFS có thể may mắn đi thẳng vào đích ngay nhánh đầu.
Câu 2	Sai	Xe đi được nhiều ô trong một nước. Nếu A và B cùng hàng cách nhau 5 ô thì chỉ cần 1 nước đi, nhưng Manhattan = 5 > 1, nên heuristic overestimate.
Câu 3	Đúng	Pacman đi theo 4 hướng với chi phí mỗi bước là 1, còn khoảng cách Euclid là đường ngắn nhất giữa hai điểm, nên không bao giờ lớn hơn chi phí thật.
Câu 4	Sai	Tổng có thể vượt chi phí thật. Ví dụ $h_1 = h_2 = h^*$, thì $h_1 + h_2 = 2h^* > h^*$.
Câu 5	Sai	Consistent ⇒ admissible chứ không ngược lại. Ví dụ ở 3.3, heuristic admissible nhưng vi phạm consistency ở cạnh C→D.
Câu 6	Sai	A* graph search cần heuristic consistent mới đảm bảo tối ưu (admissible chỉ đủ cho tree search). Phản ví dụ: S→A (1), S→B (1),

Câu	Đúng / Sai	Giải thích
		A→C (2), B→C (1), C→G (3), với h(S) = 0, h(A) = 0, h(B) = 4, h(C) = 0, h(G) = 0. Heuristic này admissible nhưng không consistent (B→C: 4 > 1 + 0). A* mở S, rồi A (f = 1), rồi C với g = 3 (f = 3). C được mở trước B (f = 5) nên bị đóng với g = 3. Khi B được mở, đường tốt hơn qua B tới C (g = 2) bị bỏ qua vì C đã đóng. Kết quả là đường S→A→C→G với chi phí 6, trong khi đường tối ưu S→B→C→G có chi phí 5.

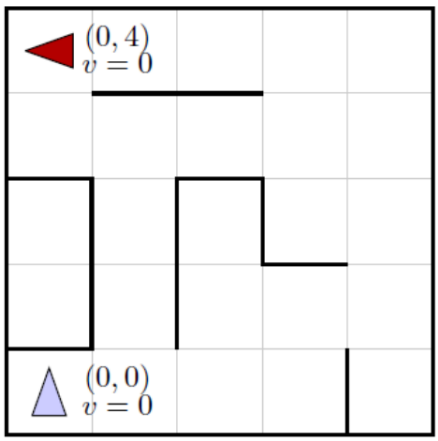
Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

- Tăng tốc (Faster): v' = v + 1
- Giảm tốc (Slower): v' = v - 1
- Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

- Rẽ trái: quay 90° ngược chiều kim đồng hồ.
- Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

- Hành động làm tác tử va vào tường.
- Làm v < 0 hoặc v > Vmax.
- Giữ tốc độ khi v = 0.
- Rẽ khi v ≠ 0.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn (v = 0) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Chuỗi hành động ngắn nhất (15 hành động): Rẽ phải, Tăng tốc, Giảm tốc, Rẽ trái, Tăng tốc, Giữ tốc độ, Giữ tốc độ, Giảm tốc, Rẽ trái, Tăng tốc, Giảm tốc, Rẽ phải, Tăng tốc, Giảm tốc, Rẽ trái.

Lộ trình: (0,0) → (1,0) → (1,3) → (0,3) → (0,4). Bắc ở (0,0) bị tường chắn nên phải rẽ phải sang Đông trước; mỗi lần đổi hướng đều phải dừng hẳn (v = 0).

Bước	**Hành động**	**Trạng thái (x, y, hướng, v)**
0	Trạng thái đầu	(0, 0, Bắc, 0)
1	Rẽ phải	(0, 0, Đông, 0)
2	Tăng tốc (v' = 1)	(1, 0, Đông, 1)
3	Giảm tốc (v' = 0)	(1, 0, Đông, 0)
4	Rẽ trái	(1, 0, Bắc, 0)
5	Tăng tốc (v' = 1)	(1, 1, Bắc, 1)
6	Giữ tốc độ (v' = 1)	(1, 2, Bắc, 1)
7	Giữ tốc độ (v' = 1)	(1, 3, Bắc, 1)
8	Giảm tốc (v' = 0)	(1, 3, Bắc, 0)
9	Rẽ trái	(1, 3, Tây, 0)
10	Tăng tốc (v' = 1)	(0, 3, Tây, 1)
11	Giảm tốc (v' = 0)	(0, 3, Tây, 0)
12	Rẽ phải	(0, 3, Bắc, 0)
13	Tăng tốc (v' = 1)	(0, 4, Bắc, 1)
14	Giảm tốc (v' = 0)	(0, 4, Bắc, 0)
15	Rẽ trái	(0, 4, Tây, 0)

b) Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

$|S| = 4 \cdot M \cdot N \cdot (V_{max} + 1)$ — có M·N vị trí (x, y), 4 hướng {Bắc, Nam, Đông, Tây} và vận tốc nhận các giá trị 0, 1, …, Vmax (Vmax + 1 giá trị).

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☒ Có ☐ Không

Ví dụ (với Vmax ≥ 2): trạng thái (1, 3, Bắc, v = 2). Phía Bắc ô (1,3) có tường ngay sát. Tăng tốc (v' = 3), giữ tốc độ (v' = 2) và giảm tốc (v' = 1) đều làm xe va vào tường; rẽ không hợp lệ vì v ≠ 0 (giảm tốc xuống v' = 0 chỉ hợp lệ khi v = 1). Vậy trạng thái này không có nút con. Trạng thái này đạt được từ (1, 1, Bắc, 1) bằng hành động Tăng tốc.

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh lớn nhất b = 3. Khi v = 0: {Tăng tốc, Rẽ trái, Rẽ phải} (không được giữ tốc độ hay giảm tốc). Khi 0 < v < Vmax: {Tăng tốc, Giữ tốc độ, Giảm tốc} (không được rẽ). Khi v = Vmax: chỉ còn {Giữ tốc độ, Giảm tốc}. Không trường hợp nào có quá 3 hành động.

Ví dụ (Vmax ≥ 2): (1, 0, Bắc, v = 1) — Tăng tốc (v' = 2, tới (1,2)), Giữ tốc độ (v' = 1, tới (1,1)), Giảm tốc (v' = 0, ở lại (1,0)). Một ví dụ khác: (1, 0, Bắc, v = 0) với {Tăng tốc, Rẽ trái, Rẽ phải}.

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☒ Không

Không chấp nhận được, vì mỗi hành động có thể đưa xe đi nhiều hơn 1 ô (v' > 1) nên số hành động thực tế có thể ít hơn khoảng cách Manhattan.

Phản ví dụ (Vmax ≥ 2, hành lang thẳng dài 7 ô, không có tường): trạng thái (0, 0, Đông, v = 0), cửa ra ở (6, 0) hướng Đông. Heuristic h = |6 − 0| + |0 − 0| = 6. Chi phí tối ưu thực tế = 5: Tăng tốc (v' = 1, x = 1), Tăng tốc (v' = 2, x = 3), Giữ tốc độ (v' = 2, x = 5), Giảm tốc (v' = 1, x = 6), Giảm tốc (v' = 0, x = 6). Vì h* = 5 < h = 6 nên Manhattan đánh giá quá cao.

Lưu ý: trong mê cung 5 × 5 ở hình, đoạn thẳng dài nhất chỉ 4 ô nên phản ví dụ được xét trên lưới tổng quát M × N (để không cố định Vmax).

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☒ Có ☐ Không

Giải thích: mỗi hành động đưa xe đi tối đa v' ≤ Vmax ô và chỉ theo một trục, nên khoảng cách Manhattan giảm tối đa Vmax sau mỗi hành động (hành động rẽ hoặc dừng thì không giảm). Do chi phí bằng số hành động nên cần ít nhất Manhattan / Vmax hành động để tới đích, tức h(s) ≤ h*(s) với mọi s; đồng thời h(đích) = 0. Vậy h là admissible.

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☒ Không

Giải thích: chi phí mỗi hành động > 0 nên g(n) tăng vô hạn dọc mọi nhánh vô hạn, trong khi h(n) hữu hạn và không âm, nên f(n) = g(n) + h(n) → ∞. Nút đích có f hữu hạn nên sẽ được mở rộng sau hữu hạn bước; tính đầy đủ không đổi.

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☒ Có ☐ Không

Có, A* có thể trả về đường đi không tối ưu. Ví dụ: cạnh S→A (1), A→G (1), S→G (3); h(S) = 0, h(A) = 10 (h*(A) = 1, đánh giá quá cao), h(G) = 0. Khi đó f(A) = 1 + 10 = 11 > f(G) = 3 nên A* mở rộng đích G theo đường S→G (chi phí 3) trước và trả về nó, trong khi đường tối ưu S→A→G có chi phí 2.

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☒ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☒ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☒ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Phương án 3 sai vì heuristic đánh giá quá cao có thể làm mất tính tối ưu (xem câu h); phương án 4 đúng vì gán h rất lớn cho một vùng sẽ khiến vùng đó không bao giờ được mở rộng.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

a) Breadth-First Search (BFS).

Bất đẳng thức:

$d(n) < d(n_s)$ (mọi nút nông hơn nút đích nông nhất đều được mở rộng)

b) Uniform-Cost Search (UCS).

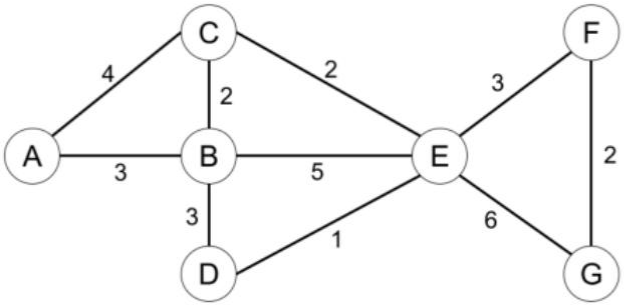
Bất đẳng thức:

$g(n) < g(n_c)$ (mọi nút có chi phí tích lũy nhỏ hơn chi phí đích tối ưu đều được mở rộng)

c) A* Search, giả sử h là heuristic nhất quán.

Bất đẳng thức:
g(n) + h(n) < g(n_c) (mọi nút có f(n) nhỏ hơn chi phí đích tối ưu đều được mở rộng)

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	**A-C-E-G**	**A-C-E-F-G**	**A-B-D-E-F-G**	**Khác**
UCS	☐	☒	☐	☐
Greedy với h1	☐	☐	☐	☒ A-B-E-G hoặc A-B-D-E-G
Greedy với h2	☒	☐	☐	☐
A* với h1	☐	☐	☒	☐
A* với h2	☐	☒	☐	☐

Giải thích nhanh: UCS luôn trả về đường tối ưu A-C-E-F-G (chi phí 11). Greedy h1: mở rộng A → B (h1 = 6) → D hoặc E (h1 = 3, hòa) → E → G (h1 = 0), nên trả về A-B-E-G (14) hoặc A-B-D-E-G (13) tùy cách phá hòa. Greedy h2: A → C (6) → E (5) → G (0), trả về A-C-E-G (12). A* h1: h1 không admissible; C có f = 4 + 9 = 13 chưa kịp mở rộng khi đích G được lấy ra với f = 12, nên trả về A-B-D-E-F-G (12). A* h2: f(B) = f(C) = 10 (hòa) nhưng mọi cách phá hòa đều dẫn tới A-C-E-F-G (11).

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án: 11 (đường A-C-E-F-G: 4 + 2 + 3 + 2 = 11)

c) Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ☒ Không Consistent: ☐ Có ☒ Không

h1(A) = 12 > h(A) = 11 (và h1(C) = 9 > h*(C) = 7) nên h1 không admissible; do consistent ⇒ admissible nên h1 cũng không consistent.*

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ☒ Có ☐ Không Consistent: ☐ Có ☒ Không

Chi phí thực h: A = 11, B = 9, C = 7, D = 6, E = 5, F = 2, G = 0; h2 ≤ h* tại mọi nút nên admissible. Nhưng h2 không consistent vì h2(A) − h2(C) = 5 > c(A,C) = 4 (tương tự A–B: 4 > 3 và E–F: 4 > 3).*

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☒ Sai

Sai: tree search không có tập explored nên DFS có thể đi vòng vô hạn theo chu trình.

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☒ Đúng ☐ Sai

Đúng: BFS duyệt theo từng mức độ sâu nên sẽ tới được nút đích ở độ sâu hữu hạn.

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☒ Đúng ☐ Sai

Đúng: giới hạn độ sâu tăng dần nên sẽ chạm tới độ sâu của nút đích.

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ☒ Sai

Sai: Graph Search chỉ nhỏ hơn hoặc bằng Tree Search (ví dụ trên cây thì hai frontier bằng nhau).

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

☐ Đúng ☒ Sai

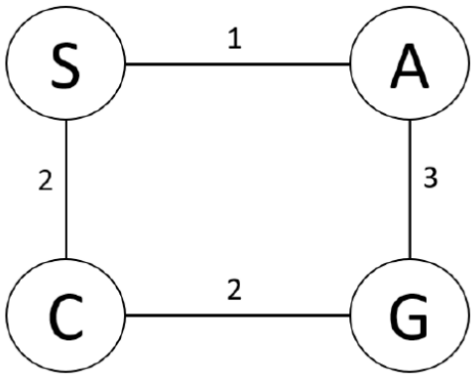
Sai: Graph Search phải lưu tập explored nên không còn bộ nhớ nhỏ như DFS.

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

☒ Đúng ☐ Sai

Đúng: với min(h1, h2) = h_i tại s', ta có h(s) ≤ h_i(s) ≤ c + h_i(s') = c + h(s').

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Trạng thái	**h(s)**
S	4
A	2
C	2
G	0

Kiểm tra: chi phí thực h*(S) = 4, h*(A) = 3, h*(C) = 2 nên h(S) = 4, h(A) = 2, h(C) = 2 đều ≤ h* (admissible). Tuy nhiên h(S) = 4 > c(S,A) + h(A) = 1 + 2 = 3 nên không consistent.